

与感知层联调

跑通感知层功能包

直线

循迹

八字绕环

跑通感知层功能包

上传的原功能包存在一些冗余的build install log文件；以及launch.py单独位于src目录下，不是很符合ROS2工程规范；model调用存在路径硬编码问题，下面上传的src已经对上述问题进行了修正，可直接编译，编译教程详见：

此处为语雀内容卡片，点击链接查看：https://dianracing.yuque.com/fsd/xw3176/boqpg5rvqp61lrzs?view=doc_embed

编译后在工作空间目录下输入命令即可运行：

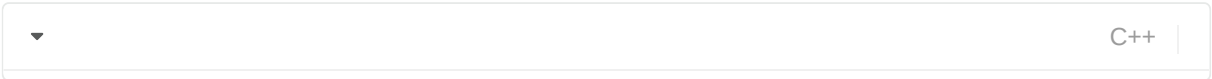
```
ros2 launch run_perception system_run.launch.py
```

[src.zip](#)

编译完后建议把setup.bash写入环境变量（以yj的路径为例，需要按照自己实际的修改路径！！）：

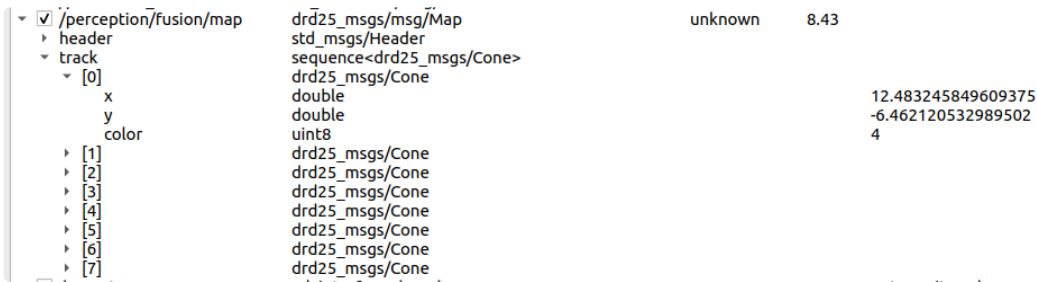
```
source ~/data/driverless26/perception_ws/install/setup.bash
```

感知层的代码结构如下：



我们需要订阅感知发布的话题 `/perception/fusion/map`

该话题包含header, track字段；其中track由多个cones[]组成，cones[]内包含x, y, color



cones[]内color的0, 1, 2, 3, 4分别对应的颜色如下：

```
src > drd25_msgs > msg > Cone.msg
1 float64 x
2 float64 y
3
4 uint8 BLUE=0
5 uint8 RED=1
6 uint8 YELLOW_BIG=2
7 uint8 YELLOW_SMALL=3
8 uint8 UNKNOWN=4
9 uint8 color
```

直线

对于直线部分的建图模块，考虑到既要求快，还要求稳的需求，我们在卡尔曼滤波的基础上，重点设计了各向异性观测噪声和分级发布策略。

1. 基于车辆坐标系的各向异性观测噪声

原本的算法是假设噪声是各向同性的，也就是认为雷达和相机在前后左右的误差都一样，像一个“圆”。但由于车辆高速行驶时，纵向测距受车速影响波动大，而横向相对稳定，这就导致它会将直线赛道的终点处的4个垂直于车头方向摆放的4个黄色锥桶粘连，强制识别成2个。

现在我们的做法是根据传感器特性构建了一个**椭球形的噪声模型**，传感器装在车上，其误差特性是跟随车头的。我们将纵向误差 σ_{long} 设置的较大（为 0.4m），而横向误差 σ_{lat} 设置的较小（为 0.2m）。而后再利用实时计算的旋转矩阵 $historical_T_veh_to_map$ ，将这个“扁长”的协方差矩阵投影到全局地图坐标系中 $R_body \ll \sigma_{long} * \sigma_{long}, 0.0, 0.0, \sigma_{lat} * \sigma_{lat}$

2. 分级发布策略

原本的算法逻辑是观测5次，等锥桶彻底稳定了再发布，但这样会导致规划层来不及规划的问题。

现在我们将地图区域划分为3个区域，第一个区域是距离车小于15米的区域，由于近处的锥桶本身设置的置信度就比较高，而且避障的优先级也最高，所以只要观测次数大于等于2，就立即发布给规划；第二个区域是15米到25米的区域，这里要求观测次数大于等于3，给滤波器一点收敛时间，防止远处的草丛或光斑干扰路径规划；最后一个区域是大于25米，要求观测次数大于等于5次且协方差收敛才发布，只留下最准确的点。

然后是关于调参的部分，主要的参数有以下几个

纵向误差 σ_{long}

横向误差 σ_{lat}

初始不确定度 $new_cone.P$

新锥桶的初始生命值 $new_cone.life$

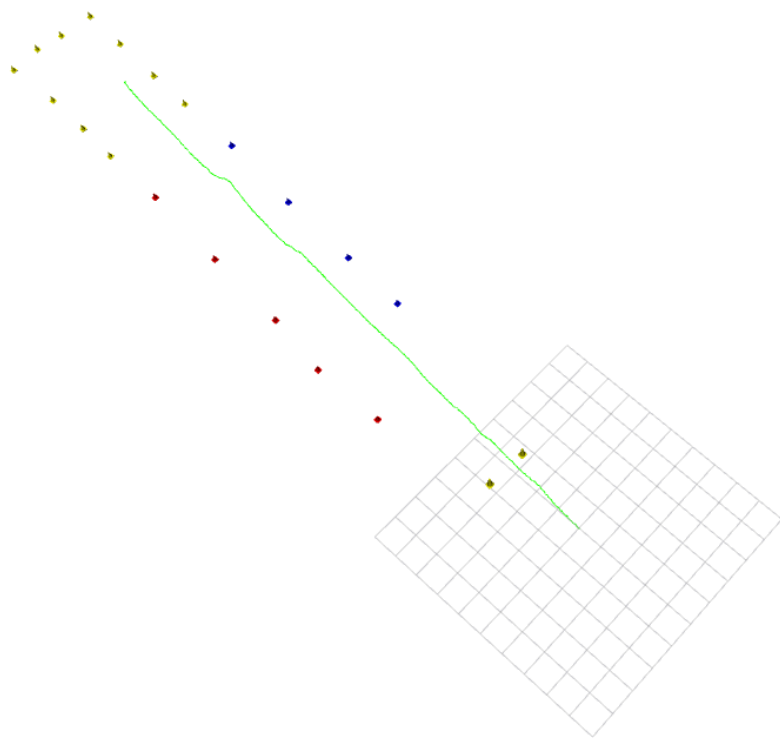
马氏距离阈值 $min_mahalanobis$

纵向误差和横向误差就不再赘述，参考上文。

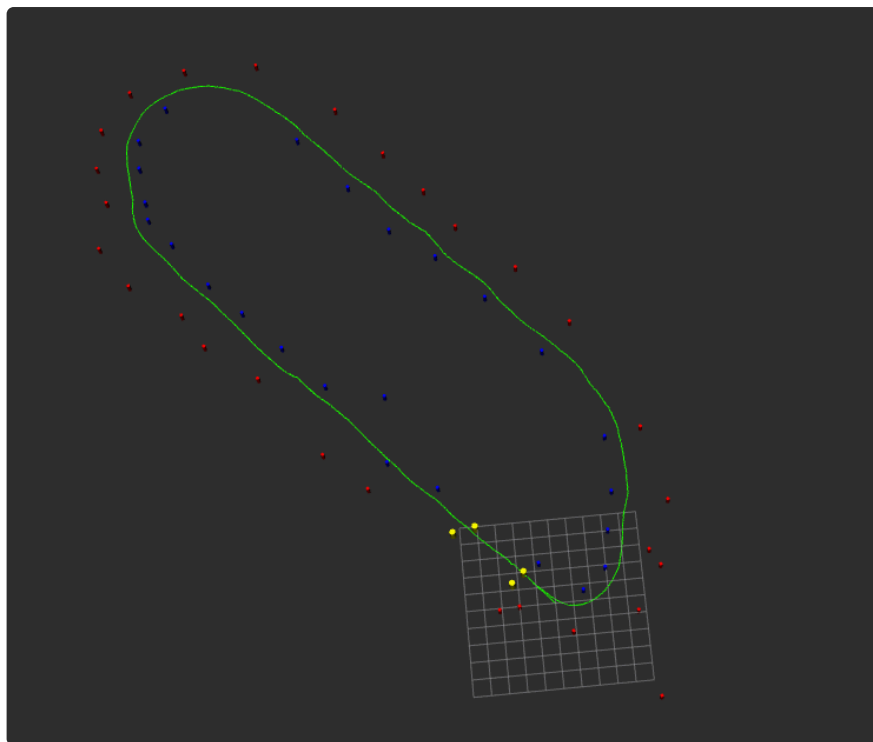
初始不确定度`new_cone.P`：我们一开始设的比较小，为0.5，导致重影的问题比较严重，现在改为了1.8。新点刚建立时，由于可能有定位漂移，我们其实并不确定它就在那。给一个大的 P 相当于给它一个“巨大的引力场”，下一帧即使观测值偏的比较大，因为它的容忍度高，也能把它吸附过来，而不是判定为新点从而产生重影。

新锥桶的初始生命值`new_cone.life`：当我们在地图里新建一个锥桶时，给它的初始生命值。每一帧如果没有匹配到，扣分。每一帧如果匹配到了，加分。分数为 0 时，从地图中彻底删除。我们给近处且清晰的点的初始生命值大一点，防止地图闪烁。给远处和未知的点的初始生命值较小，防止噪点的污染。

珏爷重构版v4.0.0同样达到了SOTA的效果



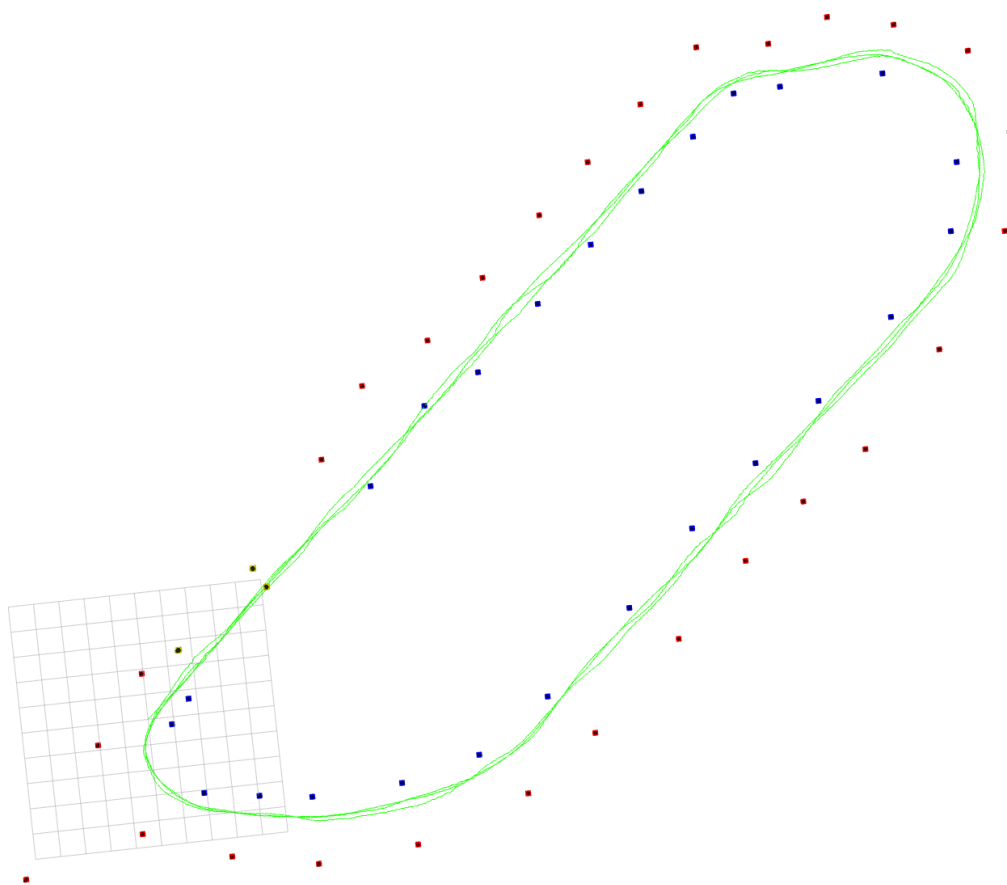
循迹



调整了大量参数，加强了非常多的观测次数限制，可以建图得到比较好的效果，不过还有问题需要改正：

1. 由于滤波更强的原因，更难完成初始帧与回环时的语义验证
2. 在比较急的弯道处观测次数较少，效果很不理想，考虑能否添加一个算法，在判断回环时对全局进行修正，排除离群噪点

v4.0.0 珏爷重构版效果，效果非常夯



八字绕环

赛事和环境的规则如下：

1. 赛道布局 (Layout)

- **形状：**赛道由两个同心圆对组成，形成一个“8”字形。
- **尺寸：**
 - 两个圆的圆心相距 **18.25米**。
 - **内圆直径：**15.25米。
 - **外圆直径：**21.25米。
 - **行车道：**位于内外圆之间，宽度为 **3.0米**。
- **标记：**
 - 内圆内侧和外圆外侧会放置锥桶 (Pylons)。
 - 地面上会有粉笔线标记圆圈位置 (注意：粉笔线在锥桶的对面，不在行车道上，车手不能压粉笔线，也不能撞锥桶)。
 - 设有入口 (Entry) 和出口 (Exit) 大门。

2. 比赛流程 (Procedure)

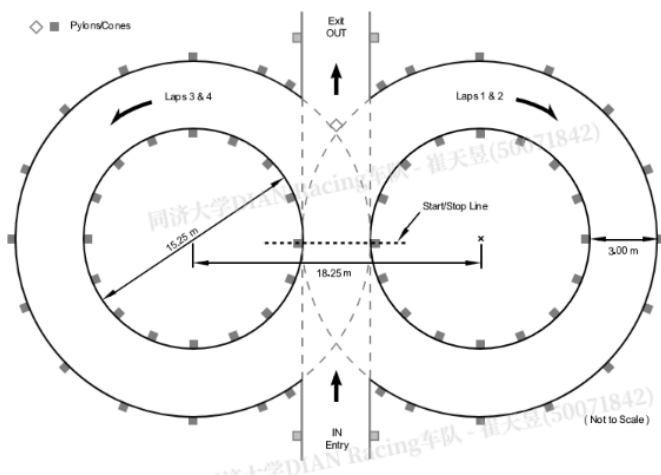
每支车队最多有 **4次** 尝试机会，可以使用 **2名** 车手，每名车手最多跑2次。

单次跑圈的具体步骤 (非常关键)：

1. **入场：** 绿旗或灯光信号亮起，车辆垂直进入“8”字赛道。
2. **第1圈 (右圆)：** 在右侧圆圈跑一整圈 (这是热身/稳定圈，**不计成绩**)。
3. **第2圈 (右圆)：** 继续在右侧圆圈跑，**这一圈开始计时** (这是右转弯成绩)。
4. **第3圈 (左圆)：** 第2圈结束后，立即进入左侧圆圈跑第三圈 (这是过渡/稳定圈，**不计成绩**)。
5. **第4圈 (左圆)：** 在左侧圆圈跑，**这一圈计时结束** (这是左转弯成绩)。
6. **出场：** 第4圈结束后，立即驶出赛道。

总结： 你实际上要跑4圈，只有****第2圈 (右弯) 和第4圈 (左弯) ****是计时的。

大体赛道如下：



颜色和锥桶要求：车行的左侧是红色，右侧是蓝色，In Entry和Ext OUT的两个锥桶是橙色的

具体程序开发中ing